Technical Architecture Specification – Gambling Levy Portal

Document Version: 2.10 | **Date:** October 2026

This document provides a comprehensive view of how the Gambling Levy Portal is built, how its components connect, and how data flows through the system. It is intended for both technical teams and business stakeholders. The domain and schema are documented separately in Data-Schema-Spec.md — this document focuses on platform architecture and API surface.

Scope note (v2.10): this specification describes the intended **production** architecture. One known case where that currently differs from the `gambling-levy-suite` repository: notifications/email (§1.3, §2.1, §3.3, §4.2, §4.3, §7) are described here as GOV.UK Notify, while `Infrastructure/Email/SendGridEmailServiceBase.cs` is the only email integration implemented in the repository today. MFA (§5.2) is the application-managed TOTP layer deployed in production, and monitoring (§1.3/§4.2/§7/§10) is implemented as described, not merely a target.

Table of Contents

- 1. Overview
  - 1.1 Purpose
  - 1.2 Users
  - 1.3 System Components
- 2. System Context (C1)
  - 2.1 External Systems
  - 2.2 User Interactions
- 3. Container Architecture (C2)
  - 3.1 Web App
  - 3.2 API
  - 3.3 Background Worker
  - 3.4 Data Stores
  - 3.5 Data Flow
  - 3.6 Component Architecture (C3)
- 4. Deployment Architecture
- 5. Security Architecture
  - 5.1 Authentication
  - 5.2 MFA Enforcement
  - 5.3 Authorisation
  - 5.4 Org Isolation
  - 5.5 Audit Logging
- 6. Data Architecture
- 7. Integration Summary
- 8. API Design
  - 8.1 Patterns

- ▸ 8.2 Endpoints
- ▸ 8.3 Error Model
- 9. Non-Functional Requirements
- 10. Observability

---

1. Overview

System Name: Gambling Levy Portal **Domain:** DHSC gambling harm prevention levy administration; Local Authority and VCSE reporting

1.1 Purpose

A secure, cloud-native, multi-org web application enabling DHSC to administer gambling harm prevention levy funding, collect three distinct data collections from ~153 Local Authorities and participating VCSE organisations, and extract cross-org analytical insights for policy reporting:

- **Annual Maturity Survey** — a living profile of each LA's strategic approach to gambling harms, confirmed annually.
- **Activities Registry** — a structured register of prevention projects each organisation is undertaking, each linked to DHSC-defined Categories, Tasks, and Key Results.
- **Quarterly Reporting Collection** — a recurring financial and performance return dynamically shaped by each org's registered Activities and their selected Key Results.

1.2 Users

The portal has exactly three roles (`Domain/ValueObjects/UserRole.cs`, consolidated under BLUM-7038 — superseding the four-Org-role / three-DHSC-role split previously described here):

Role	Constant	Description
DHSC Admin	`Role.Admin` (`"admin"`)	DHSC-only. Configures collections, forms, Key Results, organisation onboarding and cross-cutting reference data; the only role outside `AnyOrganisationMember`.
LA Admin	`Role.OrganisationAdmin` (`"organisation admin"`)	Manages users and submissions for their own organisation; member of both `AnyOrganisationMember` and `UserManagers`.
LA Lead	`Role.Lead` (`"lead"`)	Reports on their own organisation's data; reads (but cannot change) the organisation's user list. Member of `AnyOrganisationMember`, not `UserManagers`.

Composite role sets referenced throughout this document: `AnyOrganisationMember` = LA Admin + LA Lead (both LA-facing roles); `UserManagers` = DHSC Admin + LA Admin (the roles that may manage users).

> **Historical roles**: `location admin`, `contributor`, `finance` and `read only` were retired by BLUM-7038 and collapsed into `lead`. They can still appear in immutable audit-log snapshots (`HISTORICAL_ROLE_LABELS` on the client) but are never live/assignable roles.

1.3 System Components

Component	Technology	Description
Client Application	ReactJS (Azure App Service, zip deploy)	Single Page Application for all user roles
Backend API	.NET 10 / ASP.NET Core (Azure App Service, zip deploy)	REST/JSON API providing business capabilities
Background Worker	.NET 10 Worker Service — **unverified against this repo**, see §3.3 note	Processes async jobs: collection lifecycle, reminders, maturity rollovers, export generation, notifications
Database	PostgreSQL	Stores all 35 domain entities: master data, collection configuration, activities & key results, transactional submissions, longitudinal & audit records
Reporting Schema	PostgreSQL Views	Read-optimised views/aggregations for cross-org analytics & PowerBI exports
File Storage	Azure Blob Storage	Document and attachment storage
Identity Provider	Azure Entra External ID	Authentication via federated email identity
MFA	In-platform TOTP (application-managed)	RFC 6238 TOTP with recovery codes, layered on top of Entra sign-in and enforced by API middleware. Entra Conditional Access is not used — see §5.2
Monitoring	Application Insights	Metrics, exceptions and performance tracing
Notifications	GOV.UK Notify	Email delivery service

---

2. System Context (C1)

Diagram: docs/C4_Context.png · Source: docs/C4_Context.puml

2.1 External Systems

System	Integration	Purpose
Azure Entra External ID	OIDC/OAuth2	Federated email-identity authentication
GOV.UK Notify	HTTPS API	Sends collection notifications, deadline reminders, verification outcomes
External Analytics Tool	SQL Views / CSV / Parquet	PowerBI or internal BI tool consuming exported datasets

2.2 User Interactions

All users access the portal via web browser over HTTPS:

- **Reporting org users** (LA/VCSE) — authenticate with their work email, verify MFA, then complete maturity answers, register activities, and submit quarterly returns on behalf of their Org(s).
- **DHSC Super Admin** — configures collections, form versions, Key Results, org onboarding, and reminder schedules.
- **DHSC Data Verifier** — reviews submitted returns and records `accepted` / `returned` status.
- **DHSC Data Analyst** — consumes cross-org reporting views; triggers admin exports.

---

3. Container Architecture (C2)

Diagram: docs/C4_Container.png · Source: docs/C4_Container.puml

3.1 Web App (ReactJS)

- Single Page Application served to all user roles
- Loads via HTTPS from containerised hosting
- Obtains Entra ID/Access tokens via OIDC Authorization Code + PKCE (MSAL)
- After Entra sign-in, the user completes the in-platform TOTP MFA challenge and receives an MFA session cookie
- Calls APIs with Bearer JWTs plus the MFA session cookie

3.2 API (.NET 10 ASP.NET Core)

- REST/JSON API providing all business capabilities
- Validates Entra JWTs via JWKS endpoints
- Enforces MFA via middleware that checks for a valid MFA session on every non-exempt endpoint
- Enforces authorisation per endpoint (DHSC role claim + `UserOrgMembership.role`)
- Reads/writes domain data via EF Core
- Populates reporting views via SQL

3.3 Background Worker (.NET 10 Worker Service)

> **Disparity flagged, not resolved**: no standalone Worker Service project or separate deploy step exists in this repo today — `api/` has no `*Worker*` project, and the only registered `BackgroundService`/`IHostedService` is `MfaSessionRetentionHostedService`, running in-process inside the API, unrelated to the jobs below. Collection/submission status (e.g. overdue) is derived synchronously on read (`SubmissionStatusRules.Derive`), not by a scheduled job. This section may describe a planned/aspirational component, work done in a different repo, or logic that has since moved elsewhere — needs confirming with the team before treating it as current.

Processes async jobs including:

- **Collection lifecycle** — scheduled transitions `draft → scheduled → open → closed → archived`
- **Reminders** — fires `CollectionReminder` rules based on `trigger_type` (`days_before_close`, `days_after_open`, or manual)
- **Maturity rollover** — annual snapshot closure + new `MaturityRecord` creation seeded from the prior year
- **Export generation** — quarterly PDFs, yearly summary PDFs / CSVs, admin downloads; writes `SubmissionExport` records
- **Overdue tracking** — updates `CollectionOrg.status = overdue` past deadlines
- **Notifications** — sends emails via GOV.UK Notify

3.4 Data Stores

Store	Purpose
Portal Database	PostgreSQL. All 35 entities across 5 domains — see Data-Schema-Spec.md.
Reporting Schema / Views	Read-optimised views for cross-LA benchmarking, regional analysis, maturity year-on-year tracking, and exports.
Azure Blob Storage	Document and attachment storage (file-upload answers, generated export PDFs/CSVs).

3.5 Data Flow

1. User authenticates via Azure Entra External ID (federated email OTP).
2. Web App receives Entra ID/Access tokens.
3. User completes the in-platform TOTP MFA challenge; API issues an MFA session cookie.
4. Web App calls API with Bearer JWT + MFA session cookie.
5. API validates the token, enforces MFA and RBAC, processes request, persists to PostgreSQL.
6. Background Worker processes async tasks (collection lifecycle, reminders, maturity rollovers, export generation, notifications).
7. Reporting views expose datasets to external analytics tools for cross-org benchmarking and policy reporting.

3.6 Component Architecture (C3)

The API container's internal components — controllers, the MediatR command pipeline, the read-query bypass, domain aggregates, and the Infrastructure-layer services (persistence, identity, MFA, email, blob storage, telemetry) — are documented one level down in **Component-Diagram.md**, built directly from `api/src` rather than from this narrative section. GOV.UK Notify is the notification/email provider named there and throughout this spec, consistent with §1.3/§2.1/§3.3/§4.2/§4.3/§7 below (see v2.6 in the changelog).

---

4. Deployment Architecture

4.1 Azure Cloud Services

Component	Hosting	Network
Client Application (ReactJS)	Azure App Service (zip deploy via Bitbucket Pipelines)	Public (Azure Cloud)
Backend API (.NET 10)	Azure App Service (zip deploy via Bitbucket Pipelines)	Private VNET
Background Worker (.NET 10)	Docker Container — **unverified against this repo**; no separate Worker project or deploy step exists here today (see §3.3 note)	Private VNET
PostgreSQL Database	Managed Instance	Private VNET
Azure Blob Storage	Storage Account	Private VNET
Azure Entra External ID	Managed Service	HTTPS (service communication)

4.2 External Services

Service	Purpose	Protocol
Application Insights	Metrics and exceptions	HTTPS
GOV.UK Notify	Email sending service	HTTPS

4.3 Network Communication

- **Client → API**: REST over HTTPS
- **Client → Application Insights**: client-side error/metrics over HTTPS
- **API → Application Insights**: exceptions, metrics, traces over HTTPS
- **Worker → GOV.UK Notify**: email sends over HTTPS
- **API/Worker → Database**: SQL over secure connection within VNET
- **API/Worker → Blob Storage**: file operations over HTTPS (VNET-integrated)
- **API → Entra**: token validation over HTTPS

---

5. Security Architecture

5.1 Authentication

- **Identity Provider**: Azure Entra External ID
- **Protocol**: OIDC Authorization Code + PKCE
- **Sign-in method**: Federated email identity with email OTP
- **Client library**: MSAL
- **API validation**: JWKS endpoint verification of JWT signatures

5.2 MFA Enforcement

MFA is **not** enforced via Entra Conditional Access. The platform enforces an **application-managed TOTP MFA layer** on top of Entra authentication, and this is the implementation deployed in production. See MFA-Azure-AD-Limitations.md for the original rationale and MFA-Implementation.md for the build plan.

- **Factor**: RFC 6238 TOTP (6 digits, 30-second step, ±1 step tolerance, issuer `Gambling Levy Portal`), compatible with Microsoft Authenticator, Google Authenticator, Authy and other standard authenticator apps (`Infrastructure/Mfa/TotpService.cs`, `Otp.NET`)
- **Enrolment**: mandatory for every user on first sign-in; the TOTP secret is encrypted at rest with ASP.NET Core Data Protection
- **Recovery codes**: single-use, BCrypt-hashed, and can be regenerated by the user
- **Brute-force protection**: 5 failed codes lock MFA for 30 minutes and revoke every session for that user; a separate limit allows 5 attempts per minute
- **MFA session**: issued after a successful code, held in an HttpOnly / Secure cookie (`SameSite=None`, plus a `Partitioned` copy when the request is cross-site), bound to the device's User-Agent; it ends after 30 minutes of inactivity or 8 hours after sign-in, whichever comes first, and a user may hold at most 5 concurrent sessions
- **Backend enforcement**: `MfaRequiredMiddleware` refuses every non-exempt endpoint without a valid MFA session (only `/mfa/*`, `/swagger` and `/health` are exempt), so the check cannot be bypassed from the frontend
- **Admin recovery**: DHSC admins and LA Admins can reset a user's MFA, which forces re-enrolment; every MFA event is written to the audit log
- `User.mfa_status` tracks enrolment state

5.3 Authorisation

Policy-based authorisation enforced per endpoint:

- **DHSC role** — from Entra application role claim (`Super Admin`, `Data Verifier`, `Data Analyst`), or absent for reporting-org users
- **Org role** — resolved from `UserOrgMembership.role` (`lead`, `contributor`, `finance`, `read_only`) scoped by `org_id`
- **MFA strength** — checked by the MFA session middleware on every non-exempt endpoint

Example permissions (Org side):

Action	Read-only	Contributor	Finance	Lead
View org's maturity, activities, submissions	Yes	Yes	Yes	Yes
Edit maturity answers	—	Yes	—	Yes
Register / edit activities	—	Yes	—	Yes
Populate quarterly return answers	—	Yes	Yes	Yes
Submit quarterly return	—	—	—	Yes
Submit annual maturity confirmation	—	—	—	Yes
Generate yearly summary PDF (for CFO sign-off)	—	—	Yes	Yes

Example permissions (DHSC side):

Action	Super Admin	Data Verifier	Data Analyst
Configure collections, forms, KRs, reminders	Yes	—	—
Onboard orgs; edit org master data	Yes	—	—
Import statistical-neighbour data	Yes	—	—
Review submissions (accept / return)	—	Yes	—
View audit log (EventLog)	Yes	Yes	—
Access cross-org reporting views	Yes	—	Yes
Export cross-org reports	Yes	—	Yes

5.4 Org Isolation

Each Org's data is scoped at application level by `org_id`:

- Every request from a reporting-org user is scoped to the `org_id`(s) from their active `UserOrgMembership` records
- Proxy submissions explicitly populate `Submission.proxy_org_id` with the acting Org, while `Submission.org_id` remains the data-owning Org
- Application-level query filtering on all reads
- Optional PostgreSQL Row-Level Security for defence in depth
- Cross-org access only via DHSC Super Admin or Data Analyst roles

5.5 Audit Logging

Two complementary mechanisms:

- **`EventLog`** — polymorphic field-level audit across every entity (`entity_type` + `entity_id` + `field_name` + `old_value` + `new_value` + `user_id` + `change_reason`). Captures diffs automatically.
- **`KeyResultReviewLog`** — human-readable intent trail for DHSC admin actions on KRs (`created` / `updated` / `archived` / `reinstated` / `scoping_changed` + notes).

All logins, MFA events, submissions, exports, and role changes are logged. Submission records capture submitter IP / User-Agent implicitly via the MFA session binding. Structured logs are also sent to Application Insights.

---

6. Data Architecture

The full domain schema is documented in Data-Schema-Spec.md — **35 entities across 5 domains**. Summary below.

6.1 Domain Summary

Domain	Entities	Role
Master data (5)	Org, User, UserOrgMembership, Region, OrgNeighbour	Stable reference records
Collection configuration (11)	Collection, CollectionOrg, CollectionReminder, FormVersion, Section, FormSectionCategory, Question, QuestionDependency, Component, DataList, DataListItem	No-code admin definition of reporting structure
Activities & Key Results (13)	Category, Task, Activity, ActivityPartnership, ActivityKeyResult, KeyResult, KeyResultVersion, KeyResultQuestion, KeyResultCategory, KeyResultTask, KeyResultEligibility, KeyResultDependency, KeyResultReviewLog	The mechanism behind dynamic form generation
Transactional (4)	Submission, SubmissionValue, SpendReport, SubmissionExport	Submitted data, immutable once submitted
Longitudinal & audit (4)	MaturityRecord, MaturityAnswer, EventLog, Feedback	Long-term tracking and governance

6.2 Reporting Schema

Read-optimised PostgreSQL views consumed by external analytics tools (PowerBI):

- **Peer directory view** — Activities flagged `is_visible_in_directory = true`, with `narrative`-tagged values redacted
- **Maturity benchmarking view** — cross-org maturity with geographic / statistical neighbour filtering via `OrgNeighbour`
- **Financial summary view** — SpendReport aggregates by financial year and Org, plus variance against `Org.total_fund`

- **Regional aggregation view** — cross-org breakdowns by `Region`
- **Activities analysis view** — Activities joined to Category / Task / KeyResult for policy-theme slicing

6.3 Azure Blob Storage

- File-upload answers (`Component.type = file_upload`)
- Generated export documents (`quarterly_pdf`, `yearly_summary_pdf`, `yearly_summary_csv`)
- Org-scoped container / prefix patterns

6.4 Multi-Org (Data Isolation)

- Shared database with `org_id` scoping on every org-owned entity
- Application-level filtering enforced on all reads and writes
- Optional PostgreSQL Row-Level Security as defence in depth
- Proxy submissions use `proxy_org_id` (acting Org) while `org_id` remains the data-owning Org

6.5 Payment processing

Payment processing is handled **entirely offline** by DHSC finance using existing ERP systems. `SpendReport` captures reported expenditure; the portal does not trigger or record payments. The CFO sign-off for annual reconciliation is a wet-ink process — the portal generates the yearly summary PDF via `SubmissionExport` but does not ingest the signed document.

---

7. Integration Summary

Integration	Protocol	Purpose
Azure Entra External ID	OIDC/OAuth2	Federated email-identity authentication
TOTP MFA	In-platform (RFC 6238)	Multi-factor authentication enforced by the API on top of Entra authentication; works with any standard authenticator app
Application Insights	HTTPS	Exceptions, performance traces, correlation IDs
GOV.UK Notify	HTTPS API	Collection notifications, deadline reminders, verification outcomes
External Analytics (PowerBI)	SQL Views / CSV / Parquet	Cross-org reporting

---

8. API Design

8.1 Patterns

- RESTful JSON APIs
- Entra JWT Bearer authentication + in-platform MFA session cookie

- Authorisation per endpoint based on DHSC role claim and/or `UserOrgMembership.role` scoped by `org_id`
- Writes return created/updated resource representations
- Immutable resources (`Submission`, `SubmissionValue` post-submit) reject `PUT` after `submitted_at`

8.2 Endpoints

Paths below are illustrative of the intended surface. Detailed request/response schemas will be defined during Alpha.

Profile

Method	Endpoint	Role	Description
GET	`/profile/me`	Any authenticated	Current user profile, memberships, and DHSC role claim

MFA

Method	Endpoint	Role	Description
GET	`/mfa/status`	Any authenticated	Check MFA enrolment status
POST	`/mfa/setup`	Any authenticated	Begin MFA enrolment (returns QR URI)
POST	`/mfa/setup/verify`	Any authenticated	Confirm enrolment code, persist secret, issue recovery codes
POST	`/mfa/verify`	Any authenticated	Verify TOTP code, issue MFA session
POST	`/mfa/verify/recovery`	Any authenticated	Verify a single-use recovery code, issue MFA session
POST	`/mfa/recovery-codes/regenerate`	Any authenticated	Replace recovery codes
POST	`/mfa/disable`	Any authenticated	Disable MFA (requires re-verification)
POST	`/mfa/logout`	Any authenticated	End the current MFA session

Master data — Orgs (Super Admin)

Method	Endpoint	Description
GET	`/orgs`	List all orgs
GET	`/orgs/{orgId}`	Get org by ID
POST	`/orgs`	Create org
PUT	`/orgs/{orgId}`	Update org
GET	`/orgs/{orgId}/users`	List user memberships for an org
POST	`/orgs/{orgId}/users`	Invite user and create `UserOrgMembership`
PUT	`/orgs/{orgId}/users/{userId}`	Update membership (role / active)

Master data — Users (Super Admin)

Method	Endpoint	Description
GET	`/users`	List all platform users
GET	`/users/{userId}`	Get user by ID
PUT	`/users/{userId}`	Update user (is_active, contact details)

Master data — Regions & Neighbours (Super Admin)

Method	Endpoint	Description
GET	`/regions`	List regions
GET	`/orgs/{orgId}/neighbours`	Get neighbour mappings for an org
POST	`/org-neighbours/import`	Bulk import statistical-neighbour data (admin file upload)

Collection configuration — Collections (Super Admin)

Method	Endpoint	Description
GET	`/collections`	List collections (filter by type, status, financial_year)
GET	`/collections/{collectionId}`	Get collection
POST	`/collections`	Create collection
PUT	`/collections/{collectionId}`	Update collection
PUT	`/collections/{collectionId}/open`	Transition to `open`
PUT	`/collections/{collectionId}/close`	Manual close (records `manually_closed_at` / `_by`)
PUT	`/collections/{collectionId}/extend`	Extend deadline (records `extended_to` / `extended_by`)
GET	`/collections/{collectionId}/orgs`	List scoped orgs and per-org status
POST	`/collections/{collectionId}/orgs`	Add or update org scoping (supports `due_date_override`)
GET	`/collections/{collectionId}/reminders`	List reminders
POST	`/collections/{collectionId}/reminders`	Create reminder
POST	`/collections/{collectionId}/reminders/{reminderId}/send`	Fire a `manual` reminder immediately

Collection configuration — Forms (Super Admin)

Method	Endpoint	Description
GET	`/collections/{collectionId}/form-versions`	List form versions
POST	`/collections/{collectionId}/form-versions`	Create draft form version
PUT	`/form-versions/{versionId}/publish`	Publish and mark prior version `superseded_by`
GET	`/form-versions/{versionId}/sections`	List sections
POST	`/form-versions/{versionId}/sections`	Create section
PUT	`/sections/{sectionId}`	Update section
GET	`/sections/{sectionId}/questions`	List questions
POST	`/sections/{sectionId}/questions`	Create question
PUT	`/questions/{questionId}`	Update question
GET	`/questions/{questionId}/dependencies`	List conditional rules
POST	`/questions/{questionId}/dependencies`	Create conditional rule
DELETE	`/question-dependencies/{dependencyId}`	Delete conditional rule
GET	`/questions/{questionId}/components`	List components
POST	`/questions/{questionId}/components`	Create component
PUT	`/components/{componentId}`	Update component

Collection configuration — Data Lists (Super Admin)

Method	Endpoint	Description
GET	`/data-lists`	List data lists
POST	`/data-lists`	Create data list
PUT	`/data-lists/{listId}`	Update data list
GET	`/data-lists/{listId}/items`	List items
POST	`/data-lists/{listId}/items`	Create item
PUT	`/data-list-items/{itemId}`	Update item (including `is_active` soft-delete)

Activities & Key Results — DHSC curation (Super Admin)

Method	Endpoint	Description
GET	`/categories`	List categories
POST	`/categories`	Create category
GET	`/tasks`	List tasks (filter by category)
POST	`/tasks`	Create task
GET	`/key-results`	List KRs (filter by category / task / eligibility)
POST	`/key-results`	Create KR (generates initial `KeyResultVersion`)
PUT	`/key-results/{krId}`	Update KR metadata
POST	`/key-results/{krId}/versions`	Publish new version
PUT	`/key-results/{krId}/archive`	Archive (sets `archived_at`, `archived_by`, `archive_reason`)
GET	`/key-results/{krId}/questions`	List `KeyResultQuestion` mappings
POST	`/key-results/{krId}/questions`	Attach question to KR
DELETE	`/key-result-questions/{mappingId}`	Detach question from KR
GET	`/key-results/{krId}/eligibility`	List eligibility rules
POST	`/key-results/{krId}/eligibility`	Add eligibility rule
GET	`/key-results/{krId}/dependencies`	List `requires` / `excludes` dependencies
POST	`/key-results/{krId}/dependencies`	Add dependency
GET	`/key-results/{krId}/review-log`	Admin review trail

Activities — Org side (Lead, Contributor)

Method	Endpoint	Description
GET	`/orgs/{orgId}/activities`	List activities for an org
GET	`/activities/{activityId}`	Get activity
POST	`/orgs/{orgId}/activities`	Register activity
PUT	`/activities/{activityId}`	Update activity
PUT	`/activities/{activityId}/withdraw`	Soft-withdraw (status = `withdrawn`)
GET	`/activities/{activityId}/key-results`	List selected KRs
POST	`/activities/{activityId}/key-results`	Attach KR (snapshots `kr_version_id`)
PUT	`/activity-key-results/{akrId}/withdraw`	Withdraw KR from activity
GET	`/activities/directory`	Peer-visible activities (respects `is_visible_in_directory` + narrative redaction)

Activities — Partnerships (Lead)

Method	Endpoint	Description
GET	`/activities/{activityId}/partnerships`	List partnership invites
POST	`/activities/{activityId}/partnerships`	Invite partner org
PUT	`/partnerships/{partnershipId}/accept`	Partner org accepts
PUT	`/partnerships/{partnershipId}/decline`	Partner org declines

Submissions — Org side (Lead, Contributor, Finance)

Method	Endpoint	Description
GET	`/orgs/{orgId}/submissions`	List submissions
GET	`/submissions/{submissionId}`	Get submission
POST	`/orgs/{orgId}/submissions`	Create draft submission against a collection (form is dynamically generated from org's active Activities + KRs)
PUT	`/submissions/{submissionId}/values`	Upsert draft values
PUT	`/submissions/{submissionId}/submit`	Submit (captures proxy context if `proxy_org_id` supplied)

Submissions — DHSC side (Data Verifier)

Method	Endpoint	Description
GET	`/submissions`	List all submissions (filter by org / collection / status)
PUT	`/submissions/{submissionId}/accept`	Mark `accepted`
PUT	`/submissions/{submissionId}/return`	Return for changes

Spend Reports

Method	Endpoint	Role	Description
GET	`/orgs/{orgId}/spend-reports`	Lead, Finance, Verifier	List spend reports
GET	`/spend-reports/{spendReportId}`	Lead, Finance, Verifier	Get spend report
PUT	`/spend-reports/{spendReportId}/status`	Verifier	Update status (`accepted` / `queried`)

Exports

Method	Endpoint	Role	Description
POST	`/submissions/{submissionId}/exports`	Lead, Finance	Generate `quarterly_pdf`
POST	`/orgs/{orgId}/exports/yearly-summary`	Lead, Finance	Generate `yearly_summary_pdf` (for offline CFO sign-off)
POST	`/orgs/{orgId}/exports/yearly-summary-csv`	Lead, Finance, Analyst	Generate `yearly_summary_csv`
GET	`/exports`	Super Admin, Analyst	List exports (audit)
GET	`/exports/{exportId}/download`	Owner / Super Admin	Download export file

Maturity Survey (Lead, Contributor)

Method	Endpoint	Description
GET	`/orgs/{orgId}/maturity-records`	List records (year-on-year)
GET	`/maturity-records/{recordId}`	Get record
POST	`/orgs/{orgId}/maturity-records`	Create record (typically once per year)
GET	`/maturity-records/{recordId}/answers`	List answers
PUT	`/maturity-records/{recordId}/answers`	Upsert answers
PUT	`/maturity-records/{recordId}/confirm`	Annual confirmation (seeds next-year record)

Audit & Feedback

Method	Endpoint	Role	Description
GET	`/event-log`	Super Admin, Verifier	Query polymorphic audit log (filter by `entity_type` / `entity_id` / `user_id` / time range)
POST	`/feedback`	Any authenticated	Submit in-journey or post-submission feedback
GET	`/feedback`	Super Admin	List feedback for DHSC weekly triage
PUT	`/feedback/{feedbackId}/review`	Super Admin	Mark as reviewed

Reporting (Super Admin, Data Analyst)

Method	Endpoint	Description
GET	`/reporting/peer-directory`	Peer-visible activities across orgs
GET	`/reporting/maturity-benchmarking`	Cross-org maturity with neighbour filter
GET	`/reporting/financial-summary`	Allocation vs reported-spend summary
GET	`/reporting/regional-aggregation`	Breakdown by region
GET	`/reporting/activities-analysis`	Activities joined to Category / Task / KR
GET	`/reporting/export`	Export reporting dataset (CSV / Parquet)

8.3 Error Model

```
{
  "error_code": "COLLECTION_CLOSED",
  "description": "Submission rejected: the Q2 2026/27 collection closed on
2026-10-15",
  "correlation_id": "abc-123-def"
}
```

Common error codes include `MFA_REQUIRED`, `MFA_SETUP_REQUIRED`, `MFA_RATE_LIMITED`, `MFA_LOCKED_OUT`, `SESSION_IDLE_TIMEOUT`, `SESSION_EXPIRED`,

`SESSION_REVOKED`, `COLLECTION_CLOSED`, `FORM_VERSION_SUPERSEDED`, `KR_NOT_ELIGIBLE`, `PARTNERSHIP_ALREADY_INVITED`, `PROXY_NOT_PERMITTED`.

---

9. Non-Functional Requirements

Requirement	Target
Response Time	< 2 seconds typical
Availability	99.9% uptime
Scalability	Horizontal container scaling
Backups	Automated daily DB backups
Blob Retention	Soft-delete enabled
Collection cadence	Quarterly financial returns; annual maturity confirmation; activities registry updated on demand. Configurable open/close dates per collection; automated deadline reminders driven by `CollectionReminder` rules
Reporting	Quarterly returns dynamically shaped by registered Activities + Key Results; yearly summary PDF generated for offline CFO sign-off
Historic fidelity	Submissions always render against the `FormVersion` and `KeyResultVersions` in effect at submission time
Regulatory traceability	Every field-level change captured in `EventLog`; admin intent captured in `KeyResultReviewLog`

---

10. Observability

Layer	Tool	Data
Application	Application Insights	Exceptions, performance traces, correlation IDs
Frontend	Application Insights	Client-side errors, user sessions
Backend	Application Insights	API errors, slow queries, exceptions

Alerts

- HTTP 5xx error spikes
- Authentication failures (Entra + MFA)
- Query latency thresholds

- Container restarts
- Reminder / export worker job failures

Changelog

Date	Version	Changes
2026-10-29	2.10	**Corrected MFA to match production.** v2.7–v2.9 described Microsoft Entra-native MFA through Conditional Access as implemented; it is not. Production enforces the application-managed TOTP layer (`Infrastructure/Mfa`, `MfaRequiredMiddleware`), and no Conditional Access policy exists in the infrastructure. Restored the TOTP description in §1.3, §3.1, §3.2, §3.5, §5.2, §5.3, §5.5, §7, §8.1, the `/mfa/*` block in §8.2 and the MFA error codes in §8.3, updated to current behaviour (30-minute idle timeout, 8-hour cap, lockout, User-Agent session binding). MFA-Azure-AD-Limitations.md and MFA-Implementation.md are current again, not superseded. Fixed the header version, which had stayed at 2.1. Component-Diagram.md updated to match. Monitoring wording unchanged.
2026-08-10	2.9	Per team confirmation, Microsoft Entra-native MFA and Application Insights-only monitoring are implemented, not merely a target — narrowed the top-of-document scope note to the one remaining known case (GOV.UK Notify vs. `SendGridEmailServiceBase.cs`), and removed the "intended production direction… reconciling is a follow-up" framing from §5.2, which now states Entra-
2026-08-10	2.8	native MFA as fact. MFA-
25		-Azure-AD-Limitations.md/ MFA-Implementation.md are noted as describing a superseded approach, not as